Problema de los Lectores y Escritores Distribuido

Sistemas distribuidos – Práctica 2

Eduardo Guerrero – 816106

Índice

1. Introducción 3
2. Algoritmo de Ricart-Agrawala 4
2.2. Generalizado 4
2.3. Estructura de datos y representación de procesos 4
2.4. Protocolos de acceso 4
2.4.1. PreProtocol 4
2.4.2. PostProtocol 5
2.5. Peticiones Request y Repy 5
3. Modelo Actor 6
3.2. Adaptación del modelo Actor 6
3.3. Gestor ficheros 6
3.4. Diagrama de componentes 6
3.5. Diagrama de secuencia 7
4. Aspectos relevantes de la implementación 8
4.2. Uso del mutex 8
4.3. Uso de canales para coordinación asíncrona 8
4.4. Actualización de escritura 8
4.5. Gestion de arranque y finalización 8
4.5.1. Barrera de arranque 8
4.5.2. Finalización coordinada 9
4.5.3. Parada del Sistema de Mensajería 9
5. Pruebas 10
5.1. Propiedades comprobadas 10
5.2. Batería de pruebas 10
5.3. Loggin y trazas 10
5.4. Pruebas manuales 11

1. Introducción

Para esta práctica he desarrollado el problema de los lectores / escritores en distribuido, de esta forma no se necesita ningún coordinador y todos los procesos tienen la misma responsabilidad en el sistema.

Para resolver el problema he implementado el algoritmo Ricart-Agrawala generalizado en el lenguaje de programación Go.

2. Algoritmo de Ricart-Agrawala

Para implementar el algoritmo de Ricart-Agrawala en el problema hemos tenido que generalizar

2.2. Generalizado

El algoritmo de Ricart-Agrwala sirve para resolver problemas de exclusión mutua distribuida en el que todas las operaciones que pueden acceder a sección critica (SC) son de un mismo tipo.

En nuestro problema tenemos dos operaciones distintas en función del tipo de proceso que eres, las operaciones pueden ser read y write. En función de las operaciones que intentan acceder a SC se actúa de una forma u otra.

- Read-read: Si dos o mas procesos intentan acceder a SC a la vez en modo lectura el sistema lo permite\
- Read-write o Write-write: en el momento en que un proceso intenta hacer una operación de escritura es obligatorio que haya exclusión mutua para evitar condiciones de carrera.

Para ello he hecho una matriz simétrica de exclusión (exclude[a][b]) para representar que tipos de operaciones pueden ejecutarse de forma concurrente.

2.3. Estructura de datos y representación de procesos

Cada proceso está representado por la estructura RASharedDB el cual tiene un conjunto de variables que indican el estado de este mismo proceso.

- OurSeqNum (lrd): reloj de la petición actual, sirve para acceder a la SC. Cuando recibes un reloj te quedas con el mayor y sumas uno.
- HigSeqNum (clock): mayor reloj recibido de otros procesos sirve para mantener la sincronización de lrd respecto con el resto de los procesos.
- OutRepCNt (waiting_from): Contador que tiene el número de permisos que quedan para poder acceder a SC
- ReqCS (cs_stat): petición de querer entrar en SC o estar dentro de ella
- RepDefd (perm_delayed): Vector booleano donde se puede ver los procesos que se ha pospuesto su solicitud de entrar en SC.

2.4. Protocolos de acceso

Siguiendo con el algoritmo de Ricart-Agrawala cuando un proceso quiere entrar en SC es necesario que pase la operación PreProtocol y cuando quiera salir que pase por la operación PostProtocol. De esta forma mantenemos la exclusión mutua entre procesos.

2.4.1. PreProtocol

1. Baja el mutex para ejecutarlo en exclusión mutua
2. Se marca ReqCS = true se asigna el reloj OurSeqNum = HigSeqNum + 1 y se inicialia OutRepCnt = N – 1
3. Se libera el mutex
4. Se envía un mensaje (Request) al resto de procesos a través de ms

5. Se bloquea con el canal interno chrep hasta que el hilo recept de mensajes reciba N-1 respuestas (Reply)

2.4.2. PostProtocol

1. Baja el mutex para ejecutarlo en exclusión mutua
2. Se desmarca ReqCS = false y almacena el listado de todos los procesos que fueron aplazados (RepDefd[j] == true)
3. Se limpia RepDefd
4. Se libera el mutex
5. Se envía un mensaje (Reply) a cada proceso que habians sido aplazados.

2.5. Peticiones Request y Repy

Al iniciar ra se crear un hilo para recibir peticiones, las dos peticiones principales son Request y Repy

Cuando llega u mensaje Request puede ser que se responda inmediatamente con un Reply o que se posponga la respuesta marcándolo en el vector RepDefd, la condición para que un mensaje se responda instantáneo se tiene que cumplir lo siguiente:

- El exclude de la operación receptora y emisora no es true
- El proceso receptor no quiere entrar o está en SC
- Si ninguno de los anteriores casos se cumple se comparan los relojes y sino los identificadores.

3. Modelo Actor

3.2. Adaptación del modelo Actor

Para la comunicación distribuida se utiliza el modulo de mensajeria ms. Este módulo aplica los principios del Modelo Actor.

- Direccionamiento mediantes identificadores únicos (pid). La resolución de red se gestión mediante fichero de redes (users.txt)
- Comunicación asíncrona mediante buzón. Cada proceso arranca un hilo receptor en ms.New() que escucha conexiones TCP entrantes. Los mensajes entrantes se decodifican usando el paquete gob y se introducen en un canal Go con búfer
- Primitivas
    - Send(pid, msg) abre una conexión TCP hacia el destinatario, serializa el mensaje con gob y cierra la conexión.
    - Receive() extrae de forma bloqueante el siguiente mensaje del buzón mbox.

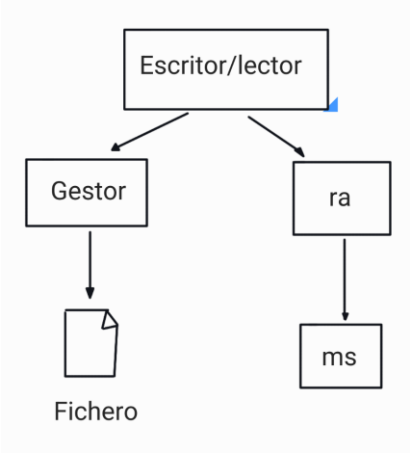
3.3. Gestor ficheros

Como intermediario entre los procesos lector/escritor y los ficheros se ha implementado un paquete gestor que encapsula el acceso al fichero de cada proceso (fichero_<pid>.txt) tiene dos funciones

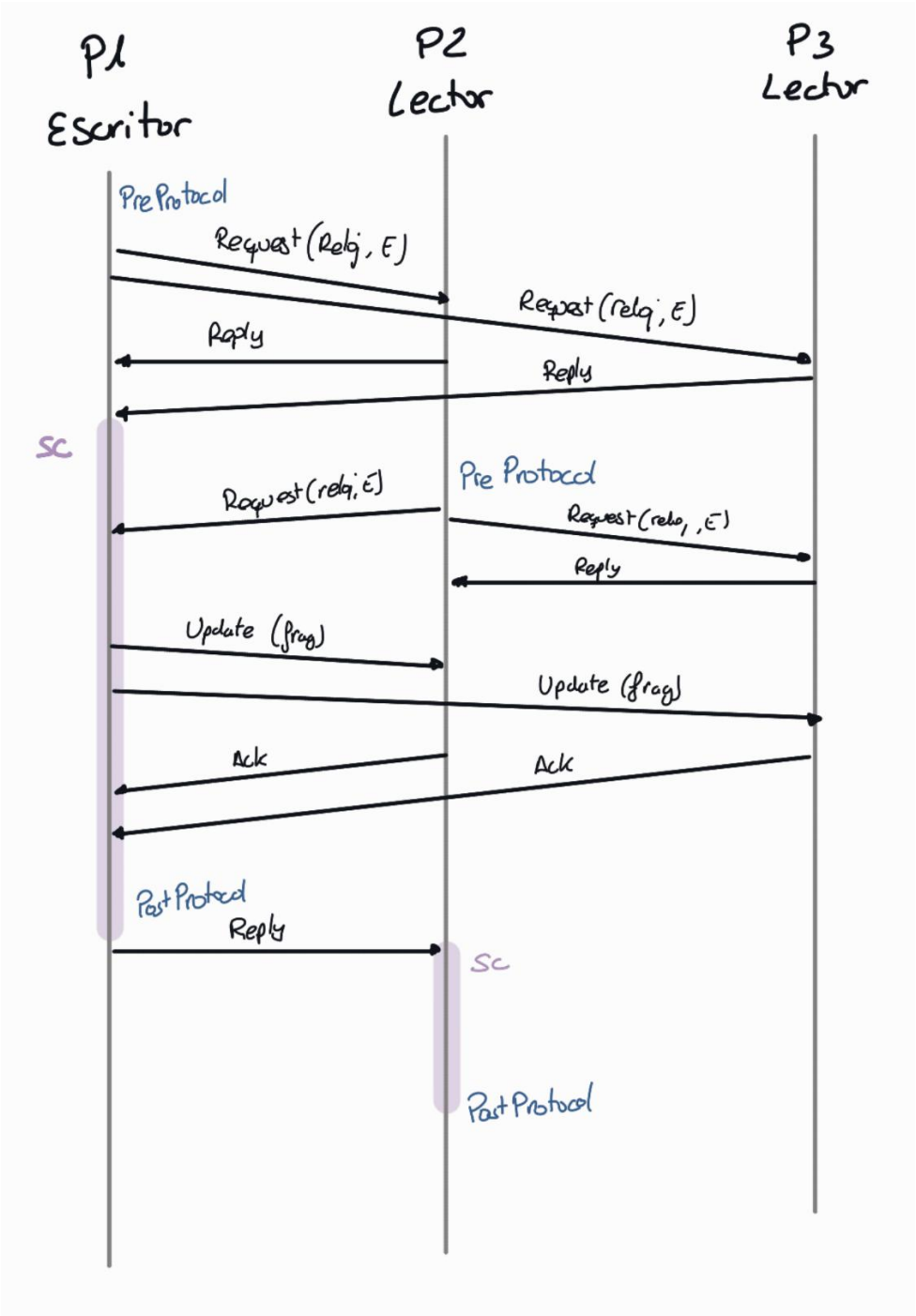
- LeerFichero(): Lee la totalidad del fichero local de texto y devuelve su contenido como cadena de caracteres. Esta operación es lanzada por los procesos lectores.
- EscribirFichero(fragmento): Añade en modo append (os.O_APPEND) una nueva línea al final del archivo. Esta operación es lanzada por los procesos escritores y por los callback onUpdate que reciben fragmentos y tienen que actualizar su fichero local

Además, tiene un mutex para garantizar que la operación es atómica

3.4. Diagrama de componentes



3.5. Diagrama de secuencia



4. Aspectos relevantes de la implementación

4.2. Uso del mutex

Dentro del modulo ra tenemos el hilo de ejecución de la aplicación, el cual llama a PreProtocol() y PostProtocol() y además tiene una goroutina asíncrona (receiveLoop) que atiende el buzón de entrada de la red.

Como estamos compartiendo variables necesitamos exclusión mutua local para evitar problemas por condiciones de carrera, esto lo hacemos mediante el puntero ra.Mutex.

Hay que proteger las variables que pueden ser modificadas dentro de ra.Mutex.Lock() (bajar el mutex) y al salir ra.Mutex.Unlock() (libera el mutex)

Para evitar interbloqueos cuando se vayan a hace llamadas se tiene que tener el mutex liberado.

4.3. Uso de canales para coordinación asíncrona

Para la sincronización de las goroutinas se utilizan canales que actúan como semáforos de señalización make(chan bool, 1). En concreto uso 3

- Chrep: notifica que se han recibido las N-1 respuestas de permiso
- chack : Notifica que se han recibido todos los Ack tras un Update
- chdone: Notifica que todos los procesos han notificado su salida (Done)

De esta forma la goroutina reciveLoop puede enviar señales (ra.chrep <- true) a un canal sin quedarse bloqueada o espera un canal (<-ra.chrep)

4.4. Actualización de escritura

Cuando un escritor escribe en su fichero tiene que actualizar el fichero del resto de procesos, esto es un problema que he solucionado con Update y Ack.

Cuando un escritor entra en la Sección Crítica, invoca la función ra.Update(fragment) para garantizar la actualización de todos los ficheros, realiza esta secuencia.

1. El escritor inicializa ra.ackCnt = N - 1 bajo exclusión mutua local y manda un mensaje Update al resto de procesos
2. Cada proceso receptor escribe el fragmento en su fichero local (onUpdate) e inmediatamente devuelve un mensaje Ack
3. El escritor se bloquea en <-ra.chack hasta que recibe todos los Ack.

4.5. Gestión de arranque y finalización

Para tener un arranque y apagado sin fallos de conexión ni accesos prematuros, se implementaron tres mecanismo de arranque y finalización.

4.5.1. Barrera de arranque

Al empezar un proceso, antes de ejecutar ningún protocolo de exclusión mutua, r.Barrier(timeout) intenta conectar y enviar un mensaje Ready a todos los

procesos de la red. La ejecución no progresa hasta que se confirma que los N procesos están activos. Esto previene de errores al iniciar los procesos.

4.5.2. Finalización coordinada

Si un proceso termina tempranamente los demás procesos cuando intenten enviarle un mensaje fallarán. Para evitarlo, al completar sus iteraciones, cada nodo ejecuta Finish()

Difunde un mensaje Done a los demás procesos si se para y mantiene activo su receiveLoop respondiendo a peticiones entrantes hasta haber recibido el mensaje Done del resto de los procesos

4.5.3. Parada del Sistema de Mensajería

Después de que Finish() confirma que todos los procesos han terminado, se llama a Stop().

Dado que ms.Stop() puede quedar bloqueado si el receptor está esperando en un listener.Accept(), la función lanza la parada en una goroutine y envía mensajes Stop al propio nodo para "despertar" la llamada bloqueante y permitir un cierre de recursos sin colgar el programa.

5. Pruebas

Para comprobar el correcto funcimiento del sistema se han realizado varias pruebas unitarias en Go (go test) además de comprobaciones manuales.

Para las pruebas unitarias cada proceso es una goroutine con su propia instancia en ra y su propio fichero (el cual se guarda en una carpeta temporal). La comunicación se realiza por TCP con el modulo ms al igual que una ejecución real.

5.1. Propiedades comprobadas

- Exclusión mutua: gracias a dos contadores atómicos que aumentan cuantos escritores y cuantos lectores están en SC podemos ver que se cumple la exclusión mutua. Un escritor no puede estar en SC junto a otro escritor o lector mientras que dos lectores si que pueden estar dentro de SC de forma simultanea.
- Consistencia en los ficheros: se comprueba que los ficheros de todos los procesos al terminar son los mismos.
- Fichero correcto: Las líneas que un escrito va escribiendo aparecen en orden y no existen repeticiones, además hay tantas como iteraciones.
- Lecturas correctas: se comprueba que un lector no lee a medias.

5.2. Batería de pruebas

Test	Que hace	Que comprueba
TEstTodosEscritores	Todos los procesos son escritores	Comprueba que hay exclusión entre escritores y se cumple le orde de escritura
TestTodosLectores	Todos los procesos son lectores	Comprueba que los lectores no se bloquean cuando no hay escritor
TestUnLectorRestoEscritores	Todos los procesos son escritores menos un lector	Comprueba que el programa funciona cuando es un único lector el que actúa junto a muchos escritores
TestUnEscritorRestoLectores	Todos los procesos son lectores menos un escritor	Comprueba que el programa funciona cuando es un único escritor el que escribe mientras hay muchos procesos que leen
TestMitadYMitad	Mitad de los procesos son escritores y la otra mitad son lectores	Comprueba que un programa funciona normal

5.3. Loggin y trazas

El programa usa el paquete log y marca que (log.Lshortfile | log.Lmicroseconds) lo que hace que cada vez que se muestra un mensaje parece la hora con microsegundos, el fichero y la línea del mensaje

5.4. Pruebas manuales

Además de las pruebas unitarias considero que las pruebas manuales son obligatorias para terminar de comprobar que un sistema funciona, por eso he realizado varias pruebas.

Pruebas	Resultado esperado
3 procesos en una máquina, cada reparto de roles entrada ysaldia	Terminan solos, ficheros idénticos
Aumentar y disminuir numero de procesos	Terminan solos, ficheros idénticos
3 máquinas virtuales, una por proceso, con users.txt con las IP reales	Terminan solos, ficheros idénticos
Procesos repartidos en desigual (2 en una máquina, 1 en otra)	Terminan solos, ficheros idénticos
Arranque en orden inverso, primero pid 3, segundo pid 2, tercero pid 1	Terminan solos, ficheros idénticos
Muchas iteraciones (100)	No se bloquean , ficheros idénticos
Un proceso con menos iteraciones que los demás (por ejemplo 1 frente a 20)	El que acaba antes sigue contestando hasta el Finish, y nadie muere
Con fichero users.txt en mal estado (vacío o faltan campos)	Mensaje de uso y salida con código 1
pid que no es un número (abc)	Mensaje de error: "el pid debe ser un número"